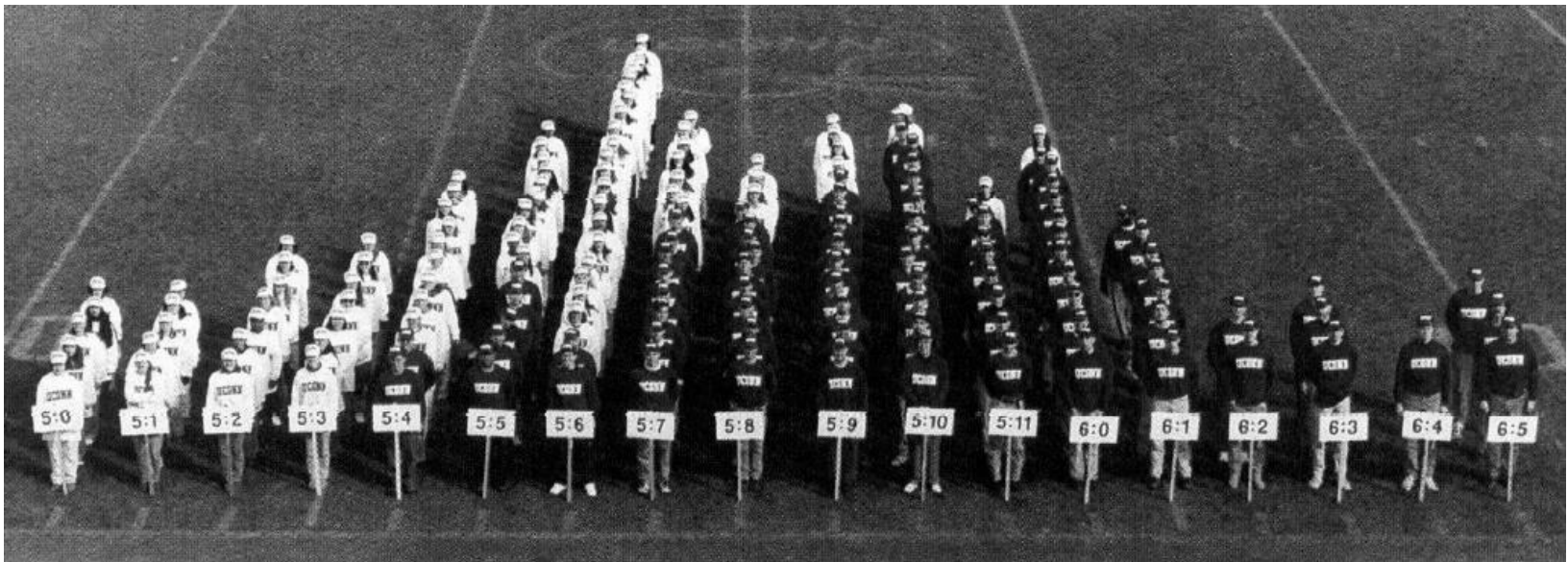


Data Visualisation and Dimensionality Reduction

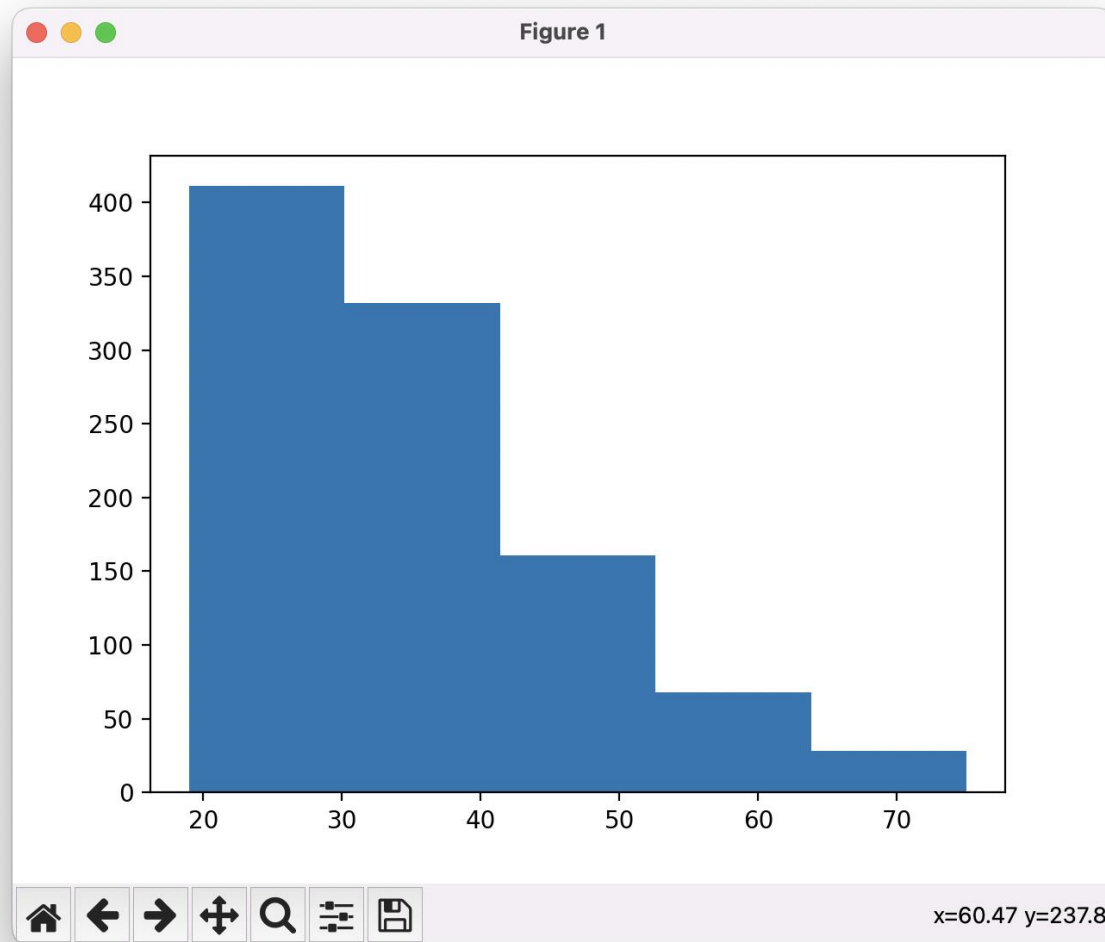
Histograms

- Estimate density in a discrete way
- Define cut points and count occurrences within bins
- How to choose cut points
 - **equal width**: cut domain (min->max) up in k equal size intervals
 - **equal height**: select k cut points such that all bins contain (approximately) n/k data points (maximises entropy)



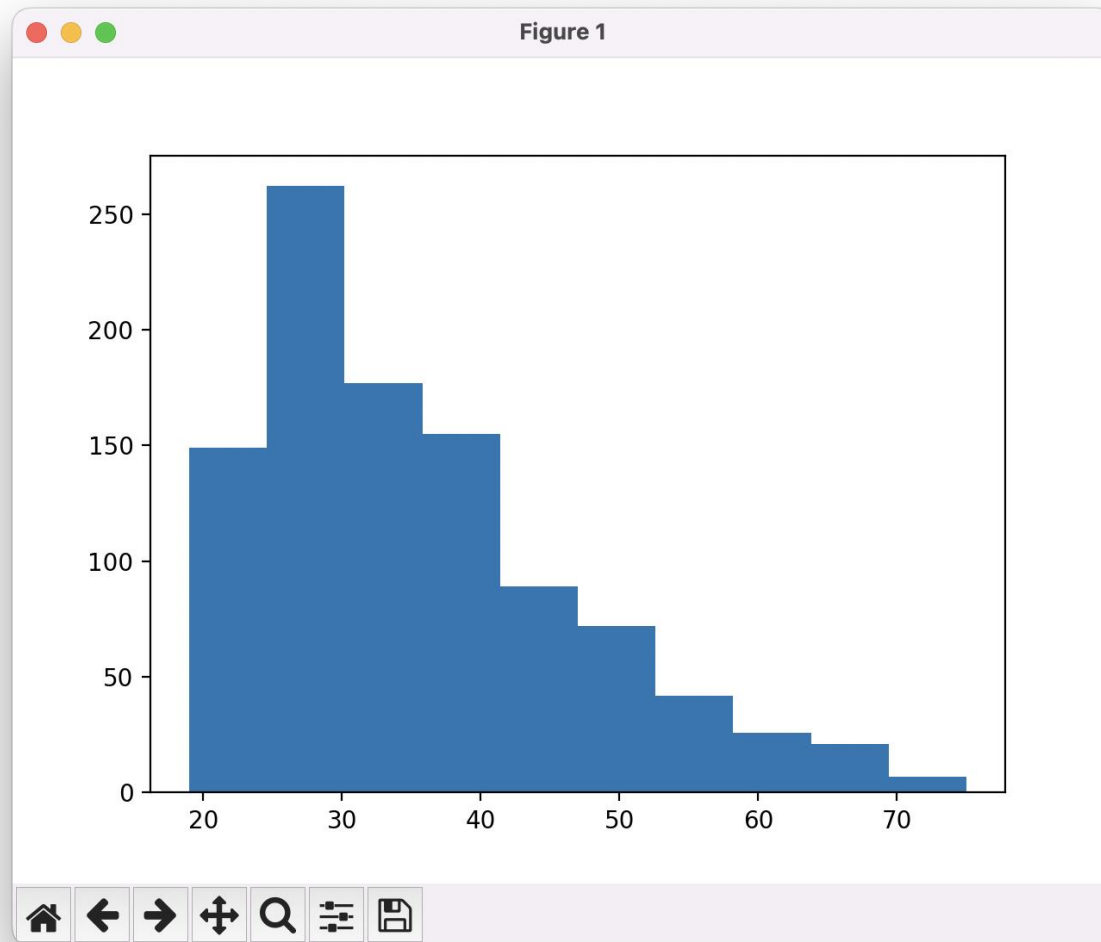
equal width

Histograms (EW, bins = 5)

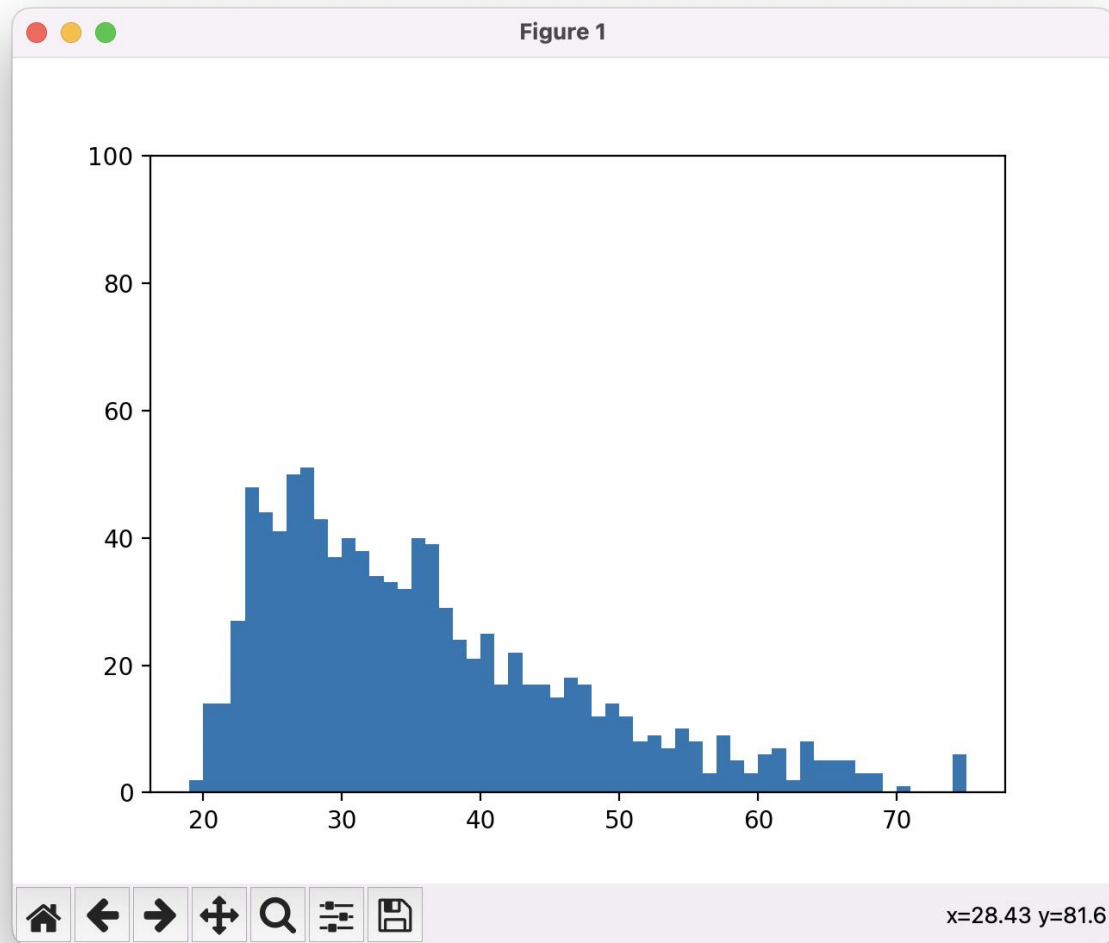


age distribution in German Credit Data (UCI)

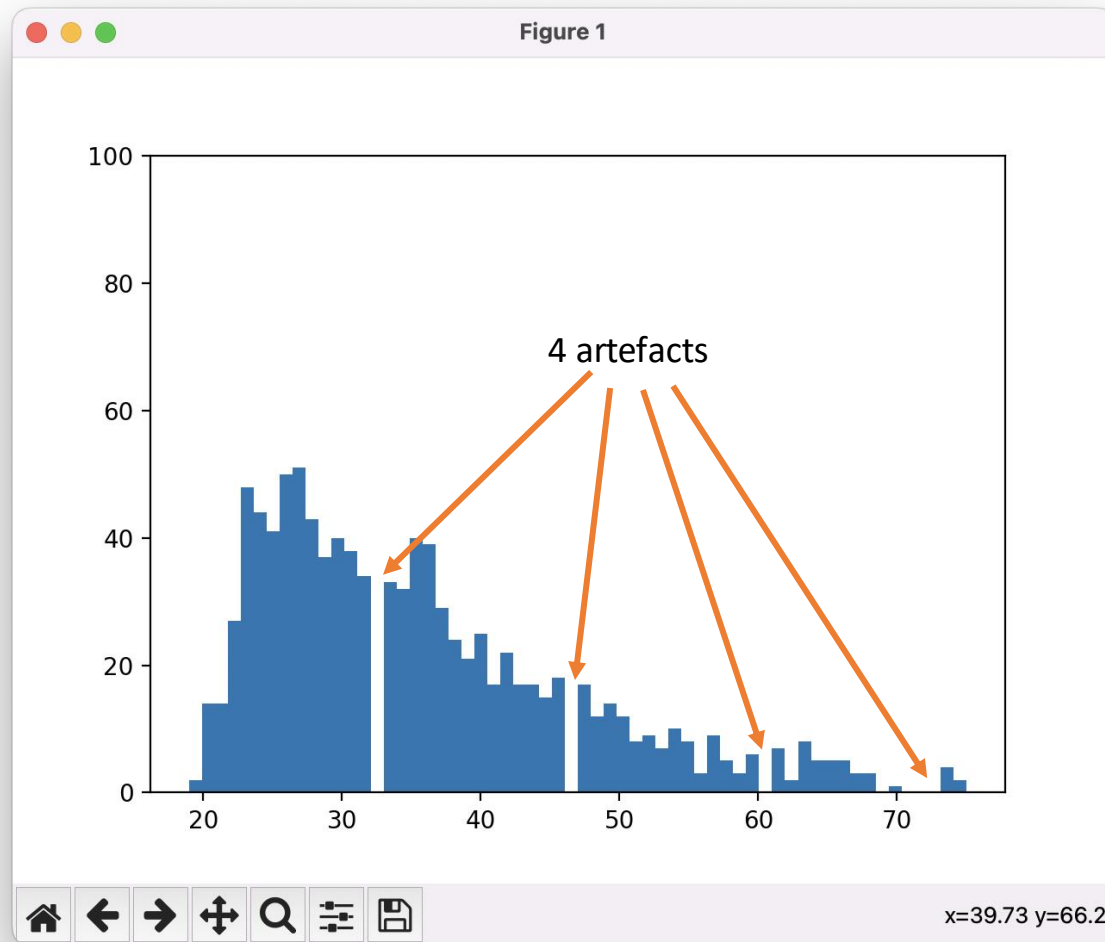
Histograms (bins = 10)



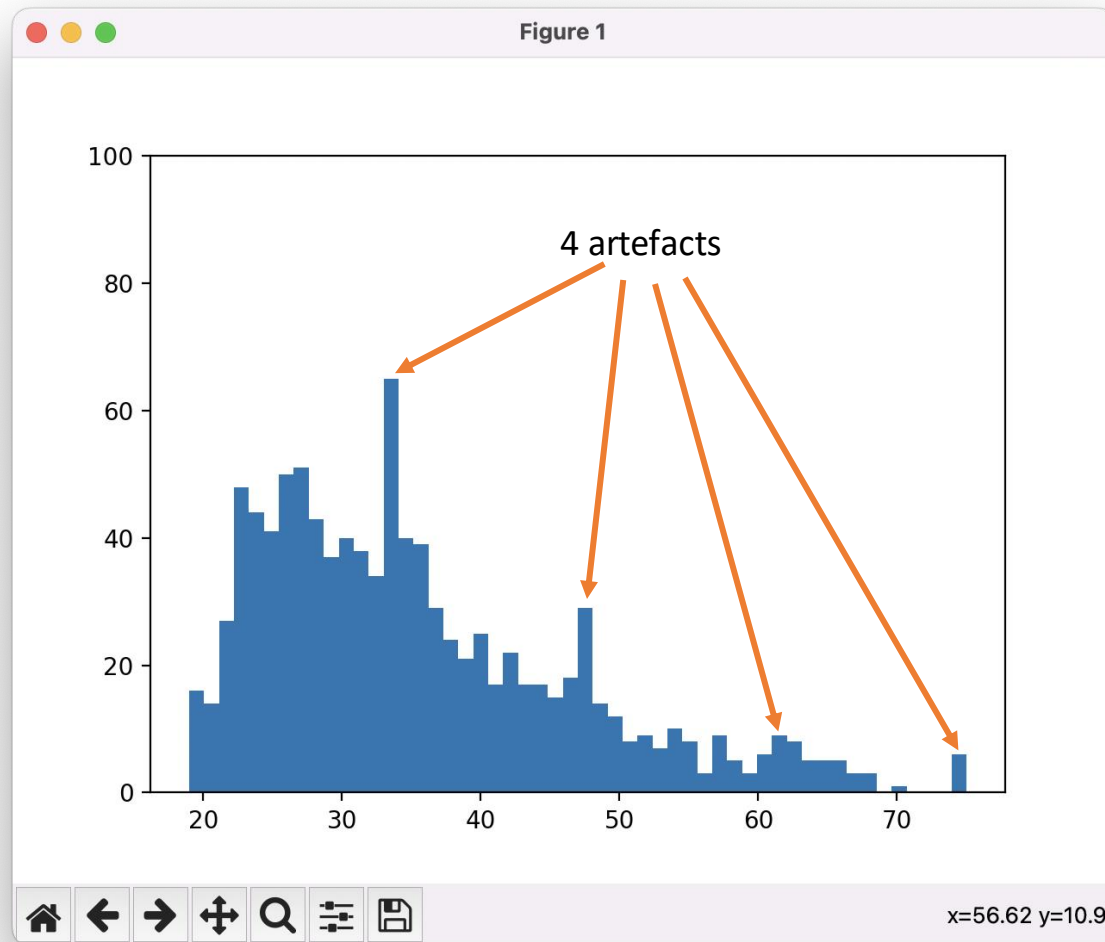
Histograms (bins = 56)



Histograms (bins = 60, +4)

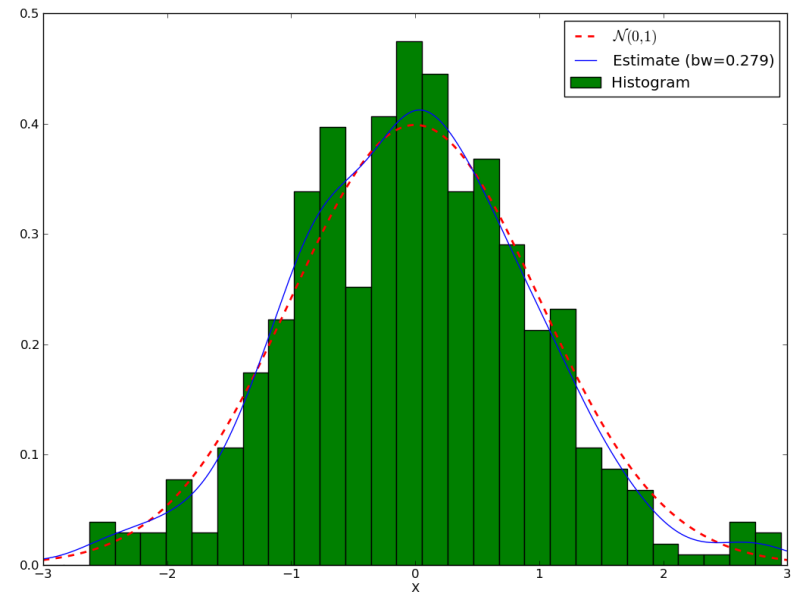
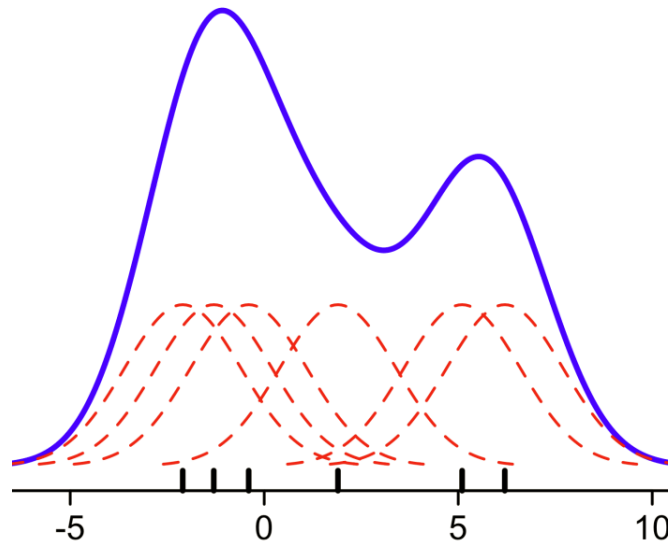


Histograms (bins = 52, -4)



Kernel Density Estimation

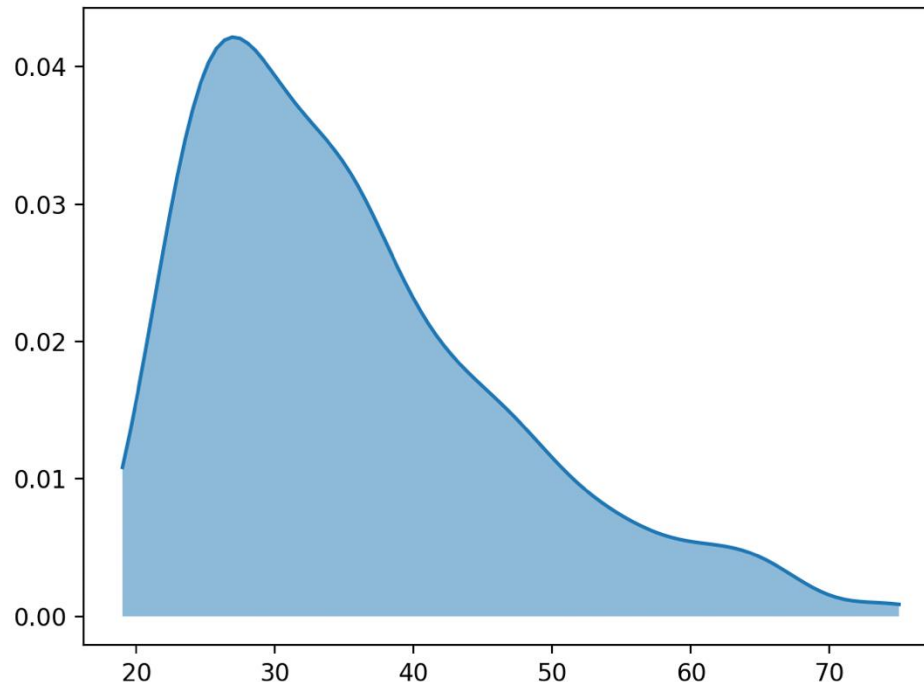
- Estimating the density (of the population) from the sample
- Observed data is smoothed over numeric domain by means of a kernel (often Gaussian)



Guessing the Right Bandwidth h in KDE

Scott's Rule: $h = \sigma \cdot n^{-1/5}$

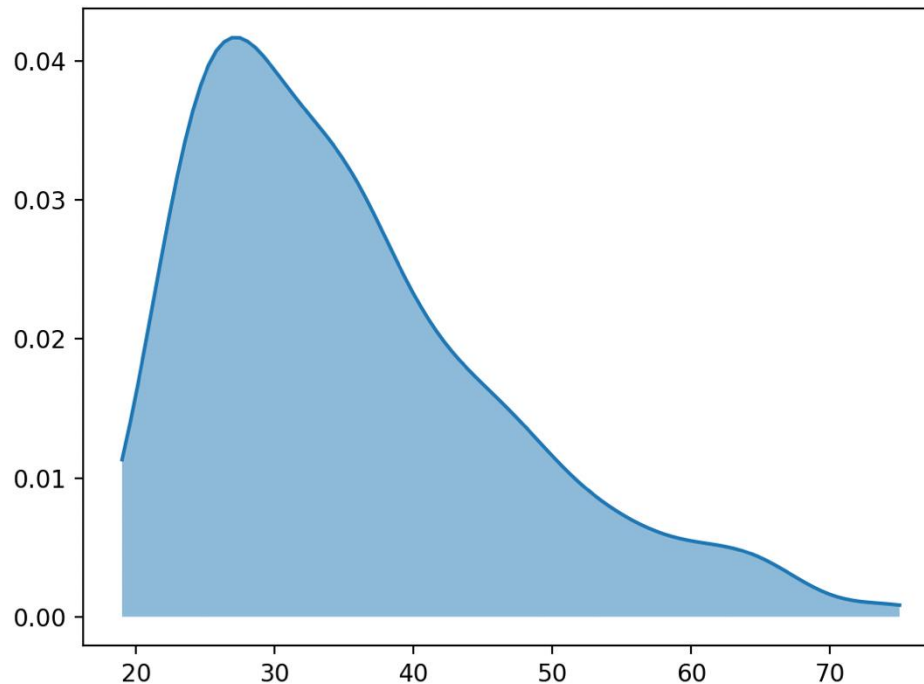
$$h = 11.37 \cdot 1000^{-1/5} = 2.854$$



Guessing the Right Bandwidth h in KDE

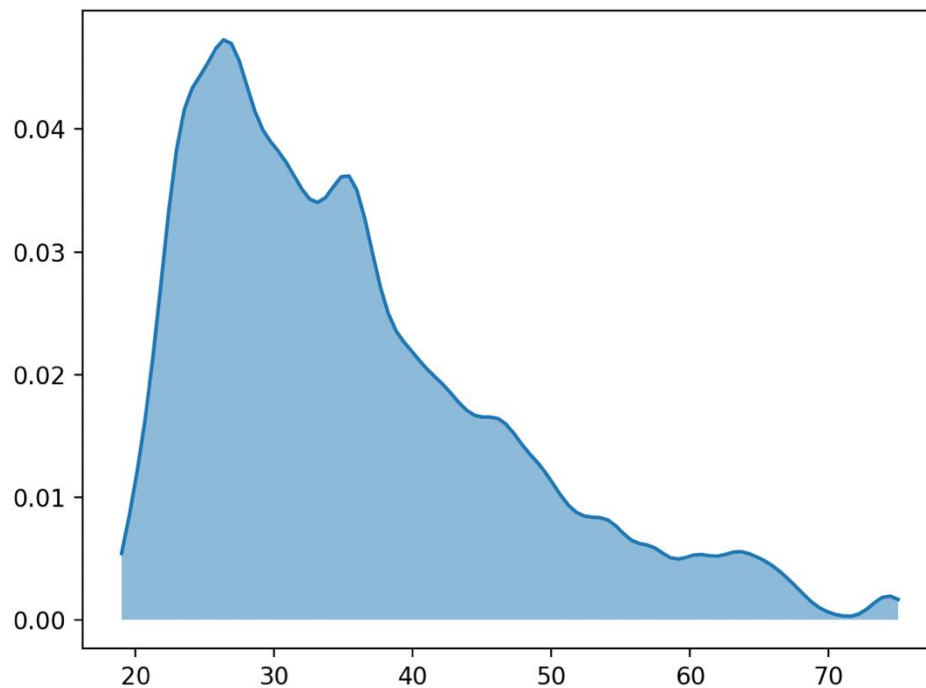
Silverman's Rule: $h = (3/4 n)^{-1/5}$

$$h = 750^{-1/5} = 0.266$$



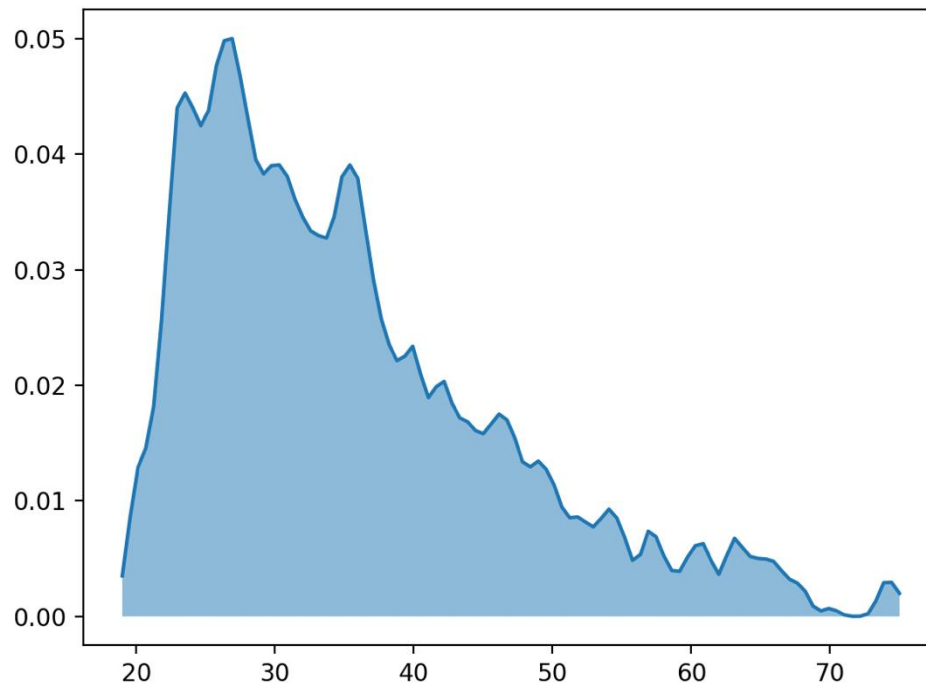
Smaller Bandwidths

$$h = 0.1$$

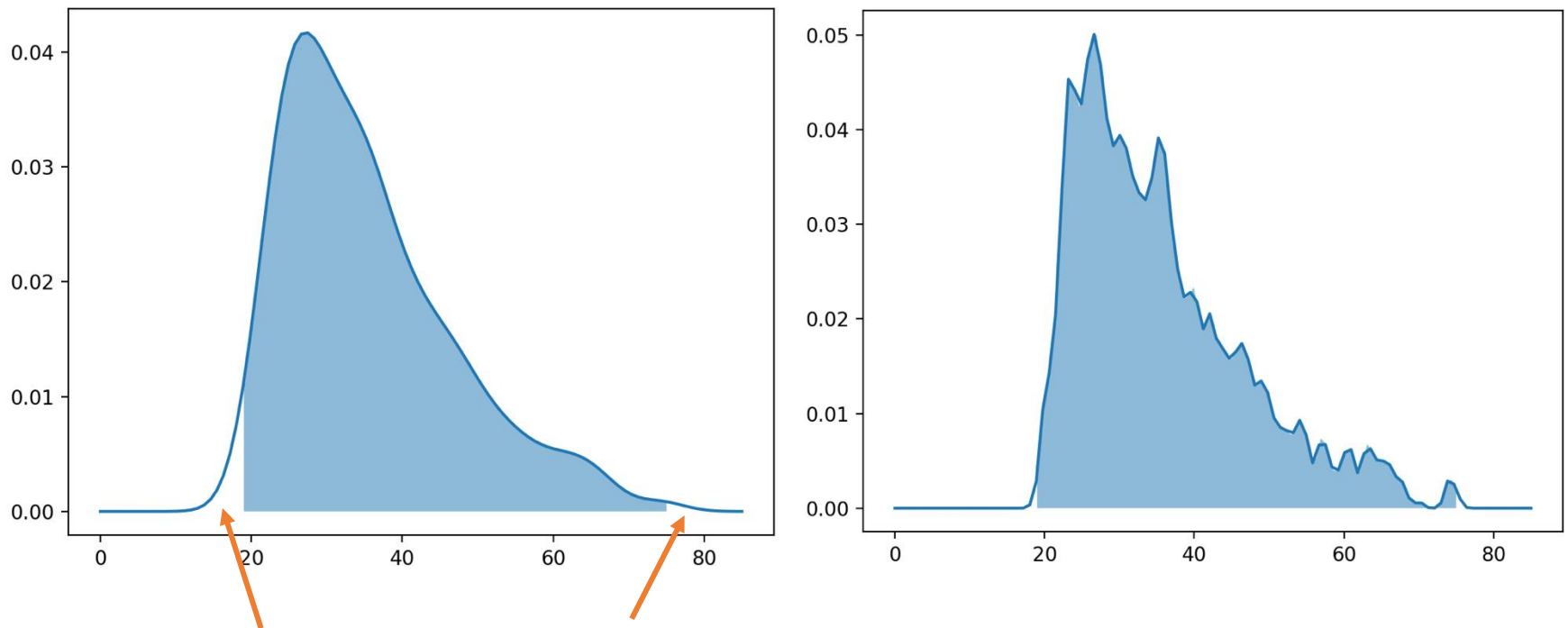


Kernel Density Estimate

$$h = 0.05$$

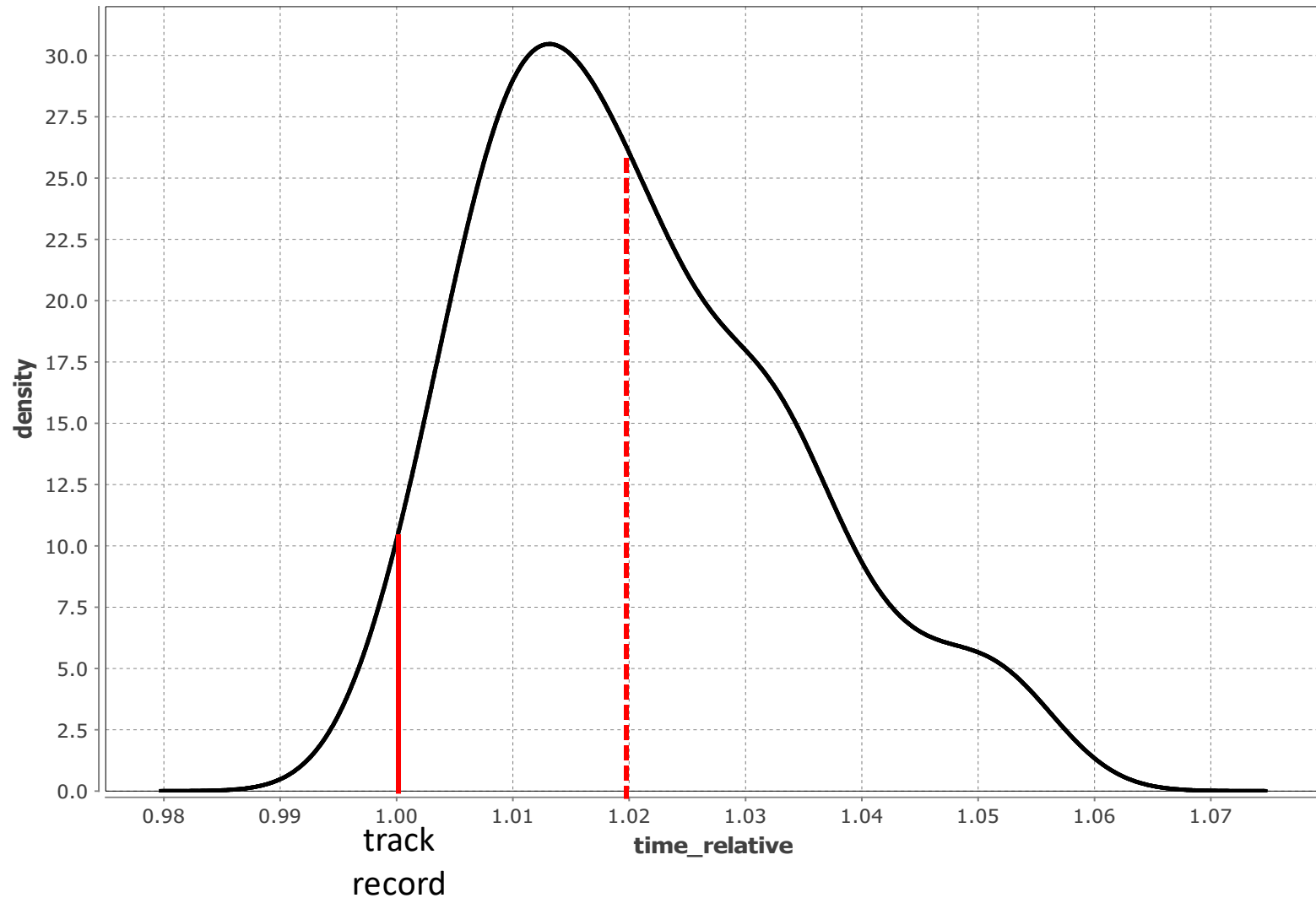


KDE Artefacts

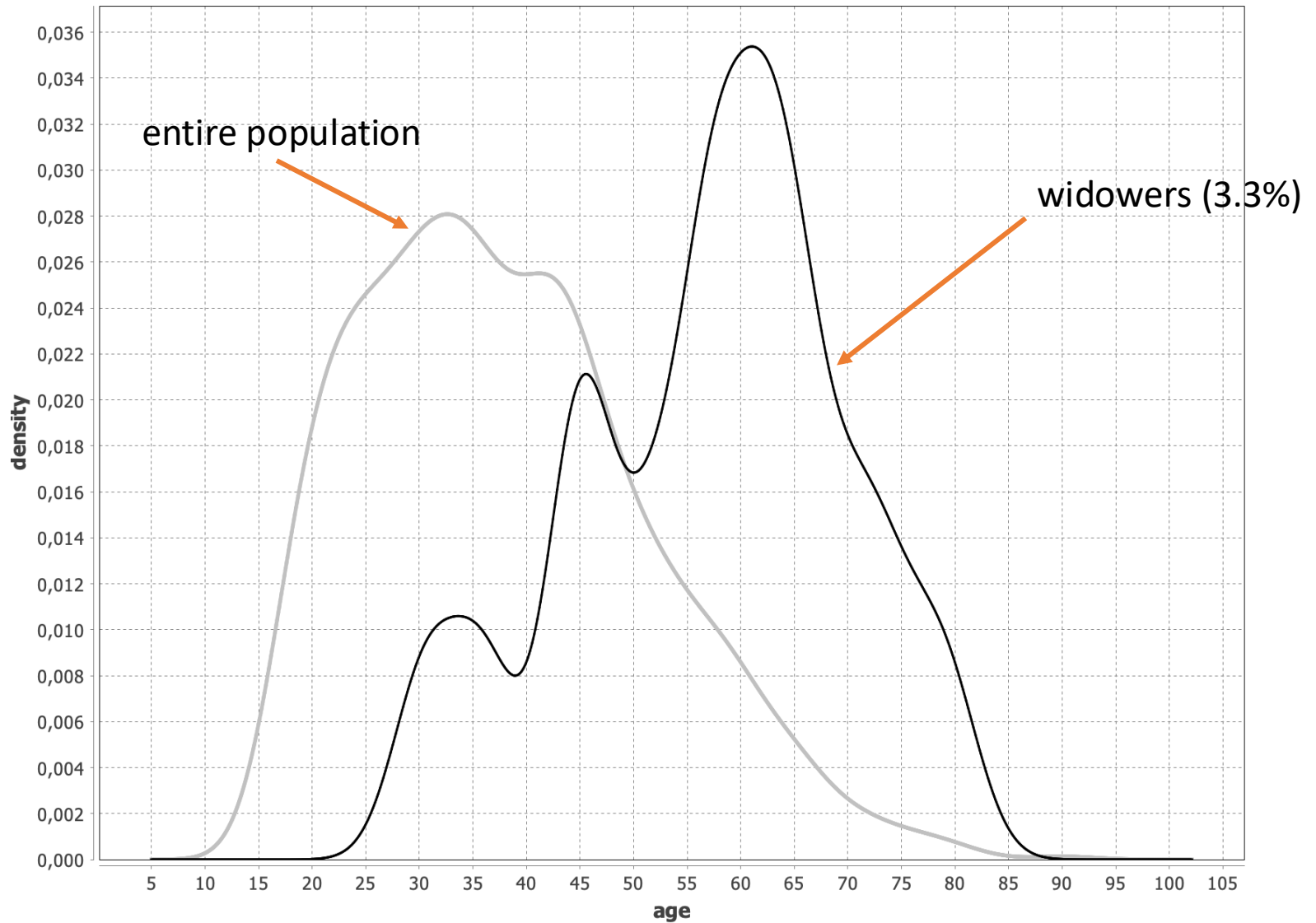


- KDE builds on the assumption that datapoints have measurement error
- Min and max in the sample are perhaps not the extreme values in the population
- KDE will be wider than the range of the data
- Size of artefact depends on bandwidth

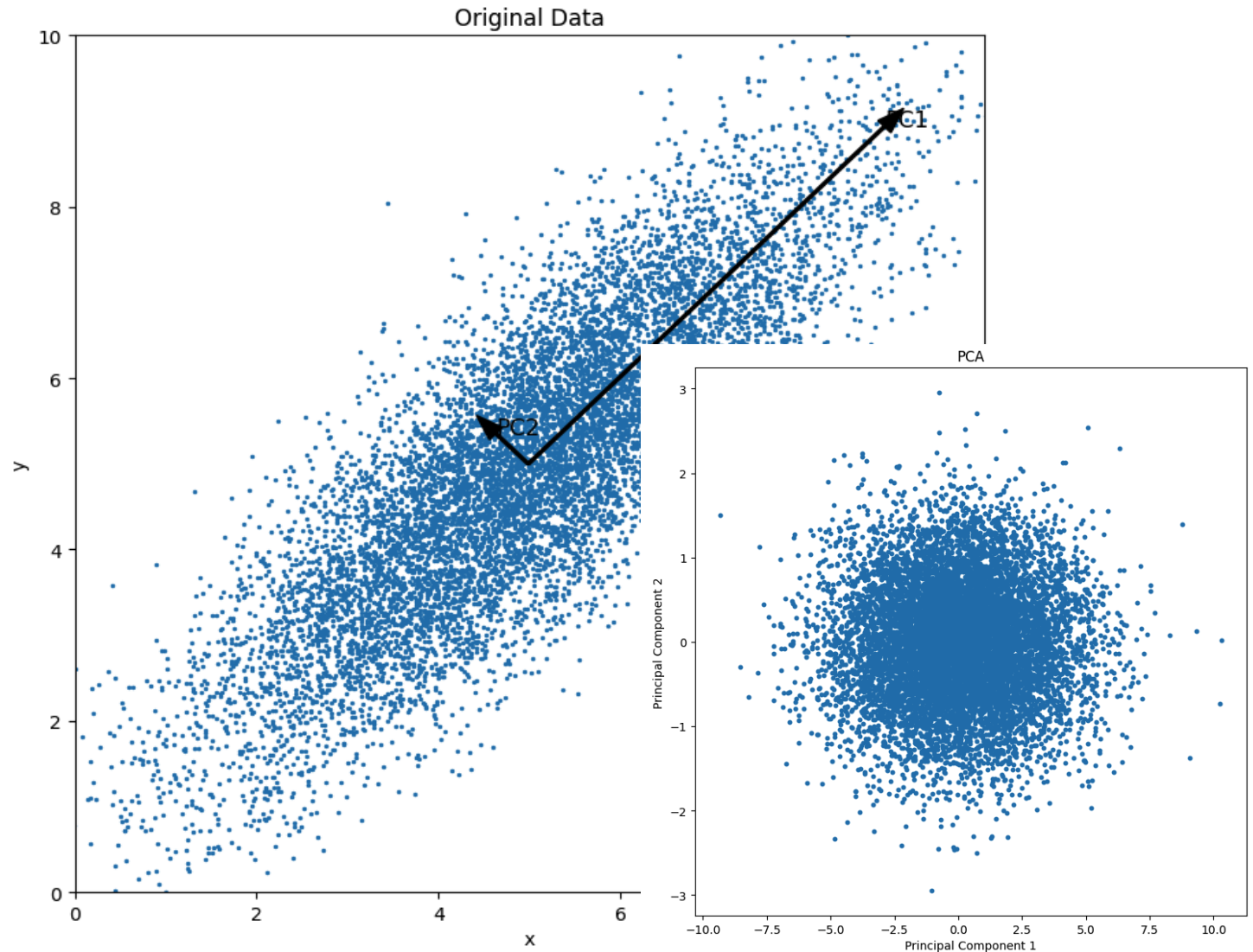
Race Times in Speed Skating



Subgroup-based KDE

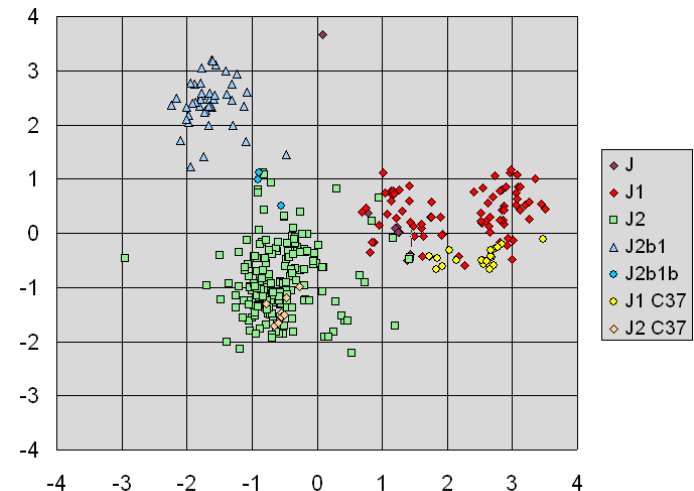


Principal Component Analysis (PCA)



Principal Component Analysis

- Linear dimensionality reduction technique
- Data projected onto a new coordinate system
 - linear combination of original features
 - principal components are orthogonal
 - components ordered by largest variance
 - first component has largest variance, second (perpendicular) component has largest remaining variance, ...
 - dimensionality reduction by selecting the first k components
- Unsupervised approach
 - not optimised for class-separability



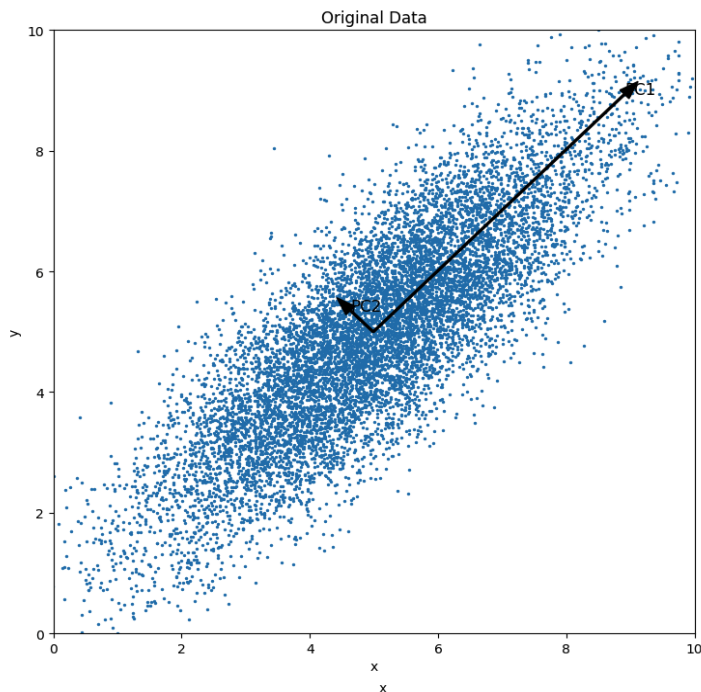
Principal Component Analysis

```
from sklearn.decomposition import PCA

mean = [5, 5]
covariance_matrix = [[3, 2.5], [2.5, 3]]
num_samples = 10000
samples = np.random.multivariate_normal(mean, covariance_matrix, num_samples)

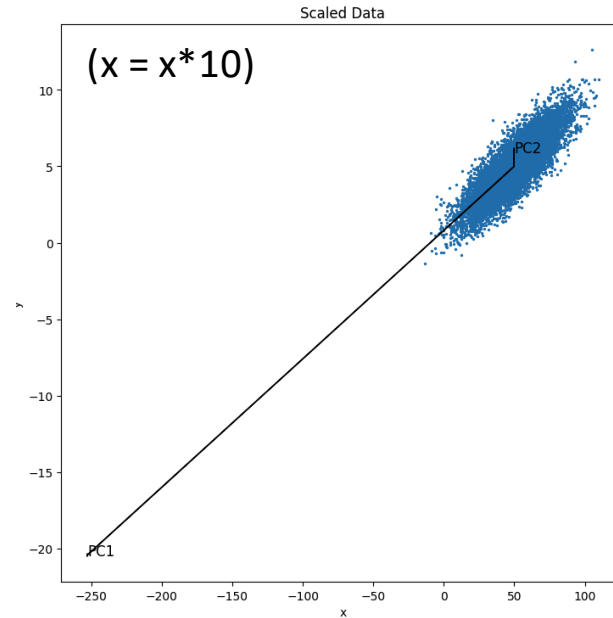
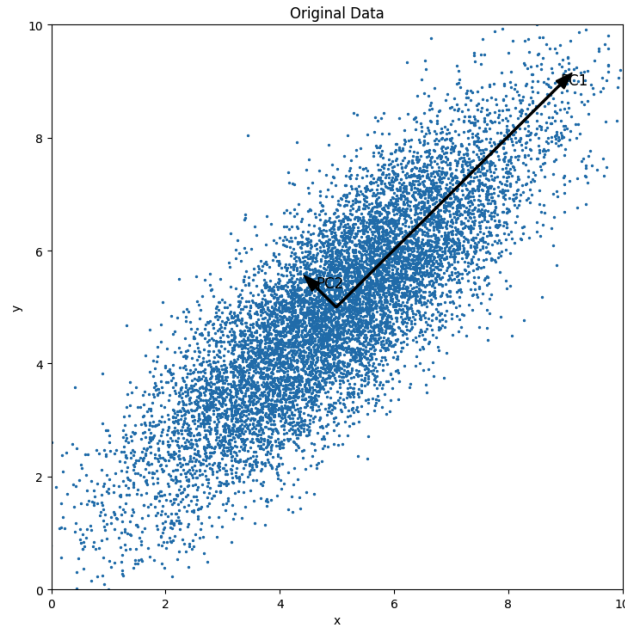
pca = PCA(n_components=2)
X_pca = pca.fit_transform(samples)

print(pca.explained_variance_)
print(pca.explained_variance_ratio_)
```



explained variance: (5.401, 0.501)
explained variance ratio: (0.917, 0.082)

PCA on Scaled Data



- When features are on different scales (e.g., one feature in *km*, another in *mm*)
- Variance of larger-scaled feature will dominate the PCA process

```
from sklearn.preprocessing import StandardScaler
```

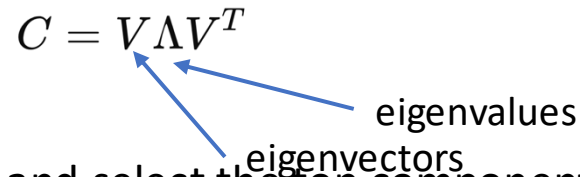
```
# With scaling (mean=0, std=1)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_pca_scaled = pca.fit_transform(X_scaled)
```

Principal Component Analysis

- (Standardize the data)
- Compute the covariance matrix C :

$$C = \frac{1}{n-1} X^T X$$

- Decompose C into its eigenvectors and eigenvalues

$$C = V \Lambda V^T$$


eigenvalues

eigenvectors

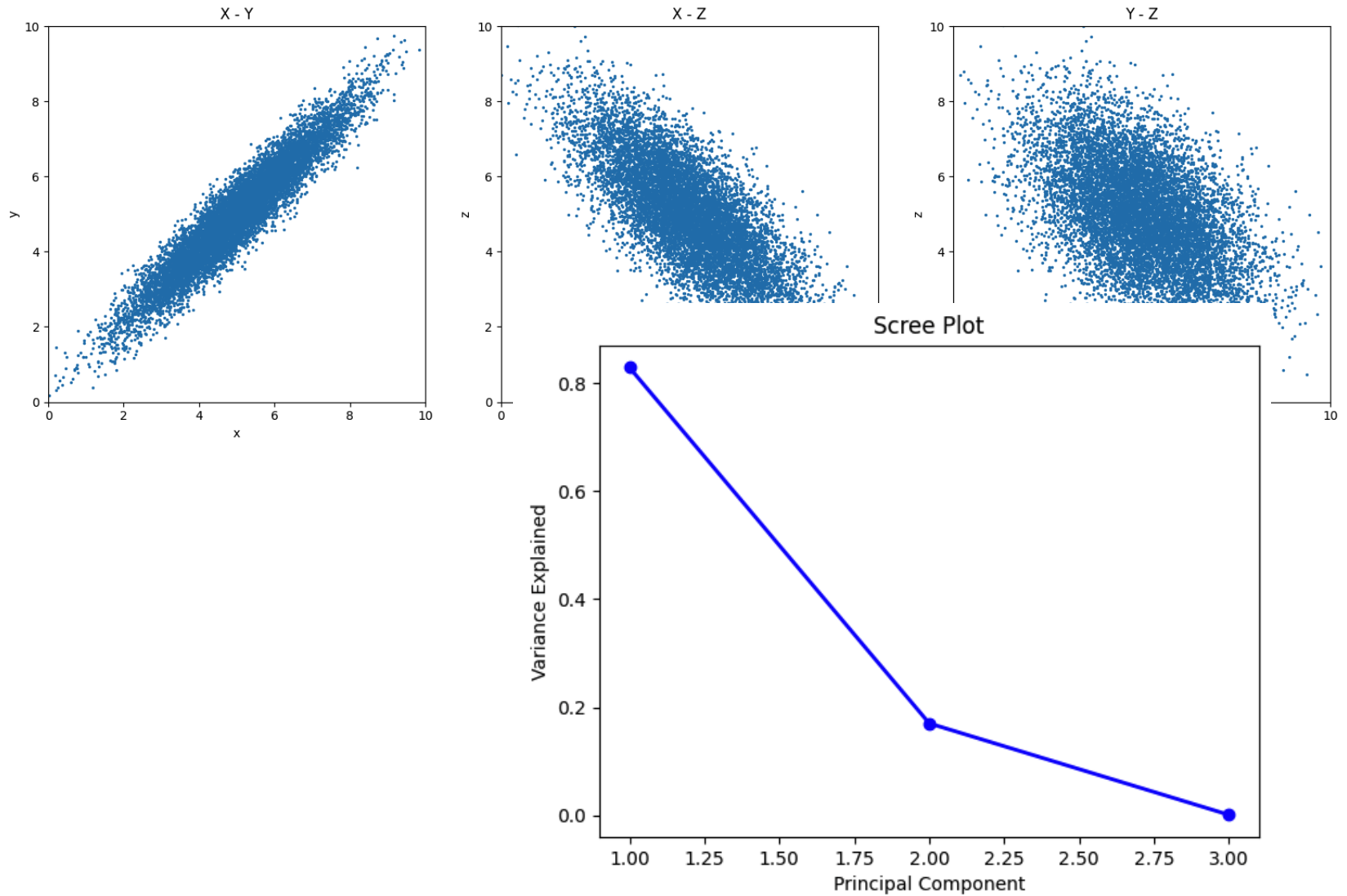
- Sort according to eigenvalues, and select the top components

- Different implementations:

- `svd_solver = 'auto'`
- `svd_solver = 'full'`
- `svd_solver = 'covariance_eigh'`
- `svd_solver = 'randomized'`

} different solvers, each with their particular reasons to choose (size of data, stability, ...)

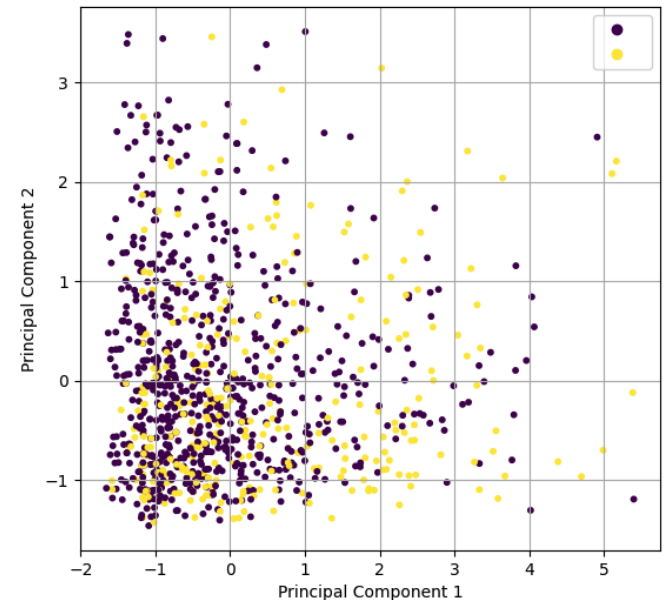
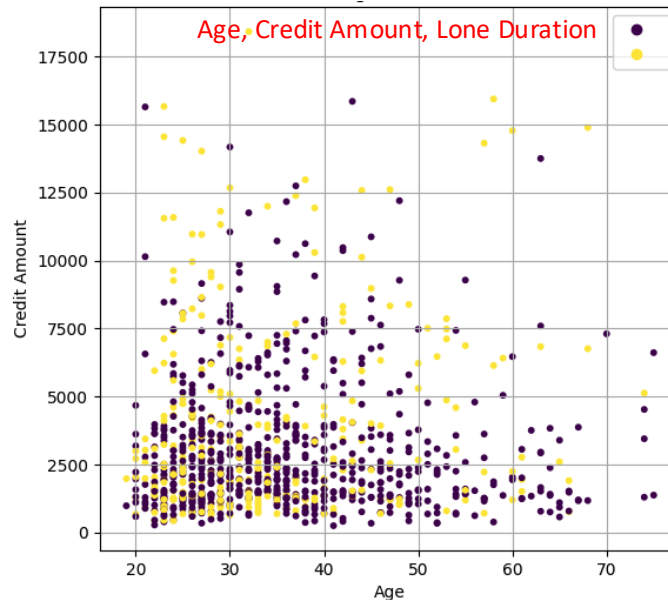
PCA Scree Plot



PCA Properties

- Results of PCA depends on scaling of variables
 - z-transform the data first
- Tacit assumptions about linearity
- Unsupervised (no target)
- Mean-removal may be annoying
- Interpretation of Principal components

German credit data

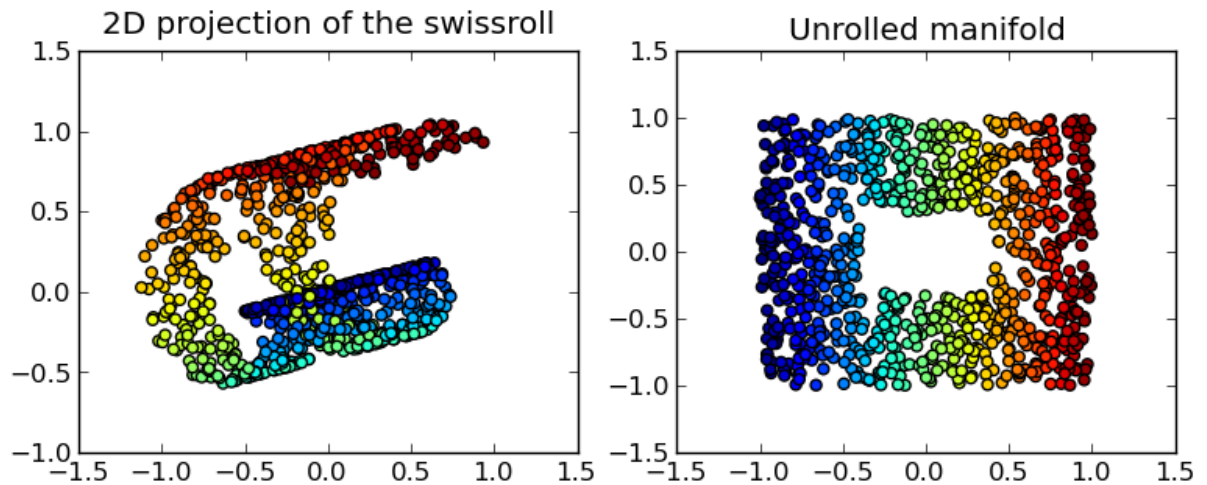


Break

t-distributed Stochastic Neighbor Embedding (t-SNE)

Non-linear Dimensionality Reduction

- Manifold
- Intrinsic lower dimensionality



t-SNE Approach

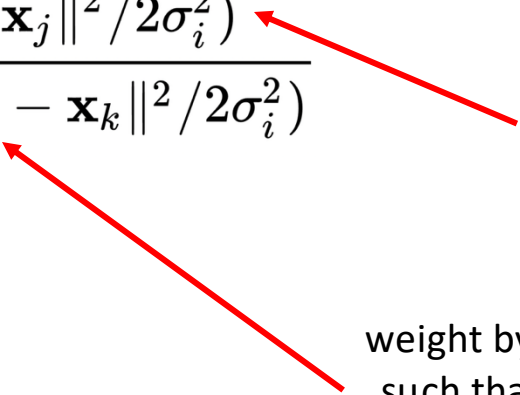
- Represent distances between every pair of datapoints
 - in the high-dimensional space
 - in the low-dimensional space
- Compute the divergence between the two
- Move the points in the low-dimensional space
 - lower the divergence

Quantify Similarity in High-dimensional Space

- Express similarity between points as probabilities

$$p_{j|i} = \frac{\exp(-\|\mathbf{x}_i - \mathbf{x}_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|\mathbf{x}_i - \mathbf{x}_k\|^2 / 2\sigma_i^2)}$$

(scaled)
Gaussian



, but $p_{i|i} = 0$

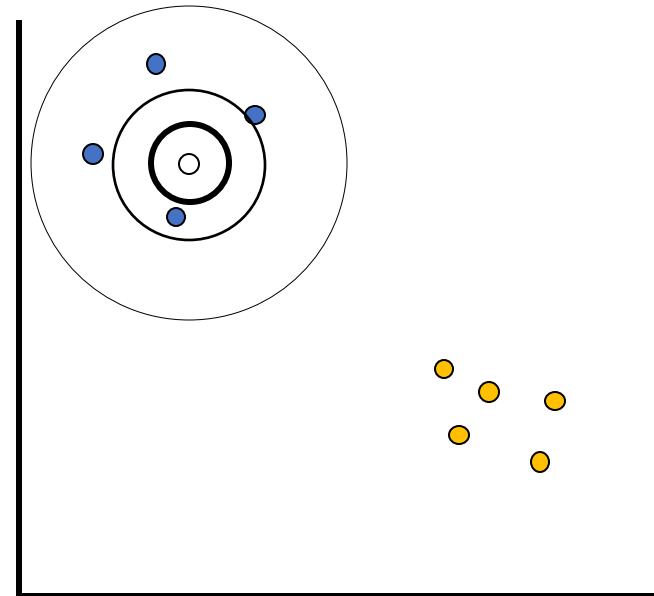
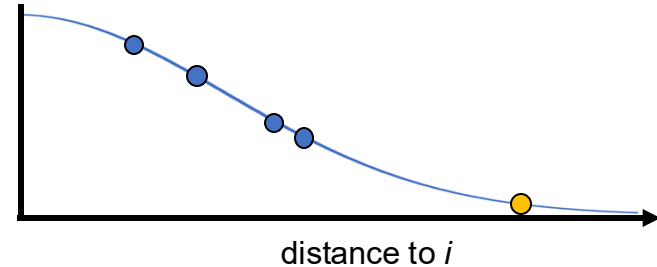
- Not symmetric, so

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2N}$$

weight by the sum,
such that sum of
 $p_{j|i}$ equals 1

Gaussian around Point i

- Gaussian captures notion of neighbourhood
- Nearby points get high probability, far away diminish to 0



Quantify Similarity in Low-dimensional Target Space

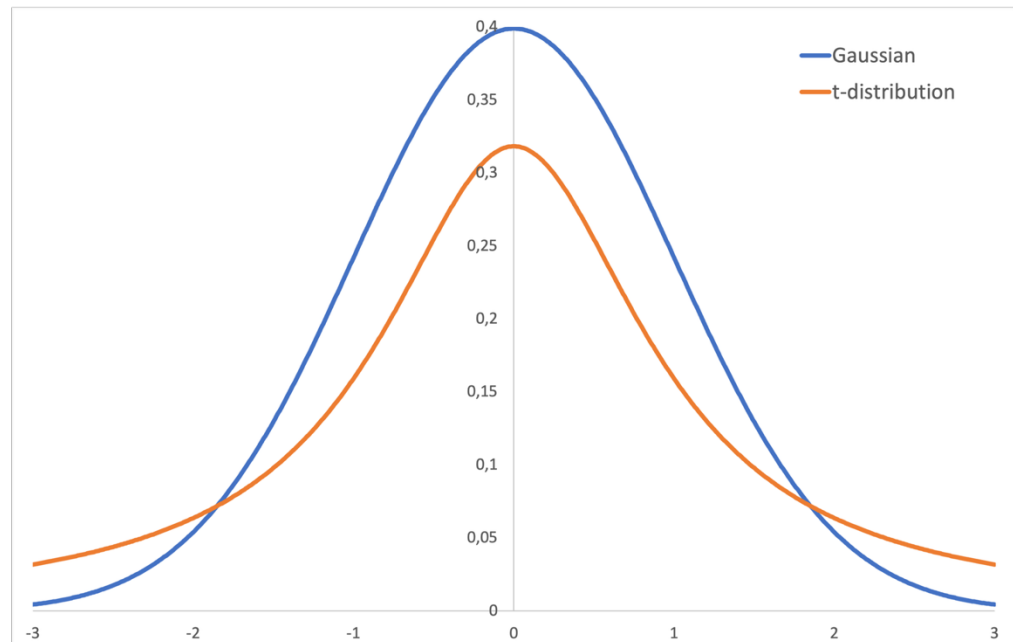
- Student's t -distribution, heavy-tailed

$$q_{ij} = \frac{(1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2)^{-1}}{\sum_k \sum_{l \neq k} (1 + \|\mathbf{y}_k - \mathbf{y}_l\|^2)^{-1}}$$

- Improvement of t-SNE compared to original SNE

Gaussian vs. t-Distribution

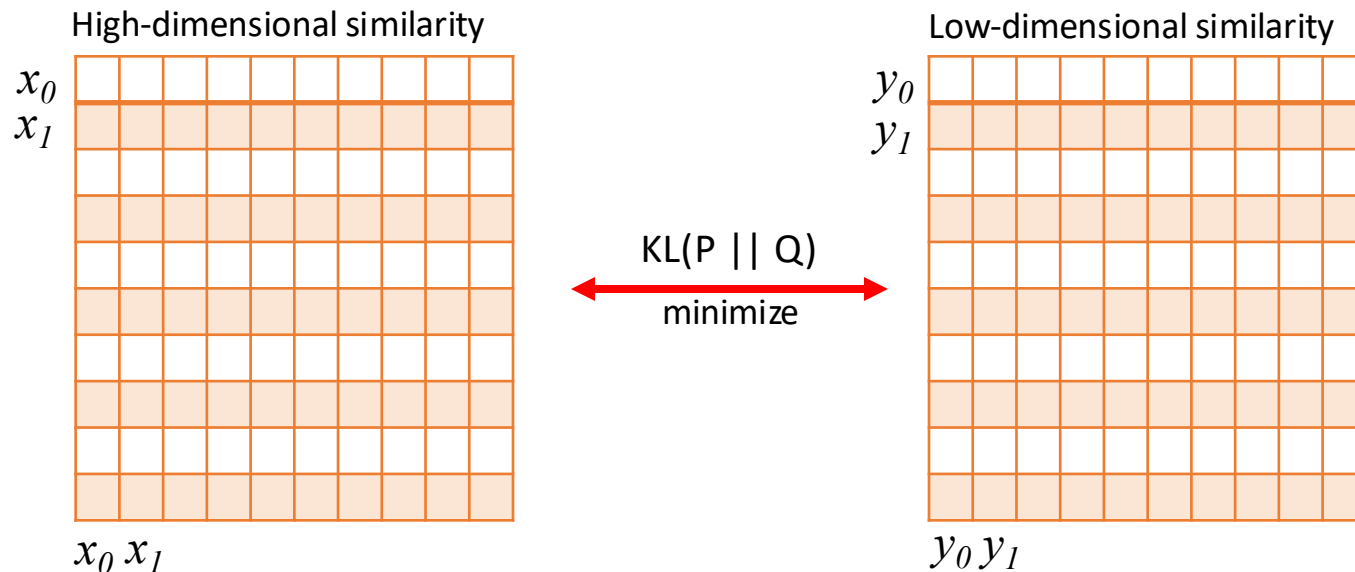
- Gaussian: $\frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$
- t-Distribution (1 degree of freedom): $\frac{1}{\pi(1+t^2)}$
- Similar bell-shaped, symmetric curve
- T-distribution has a thicker tail



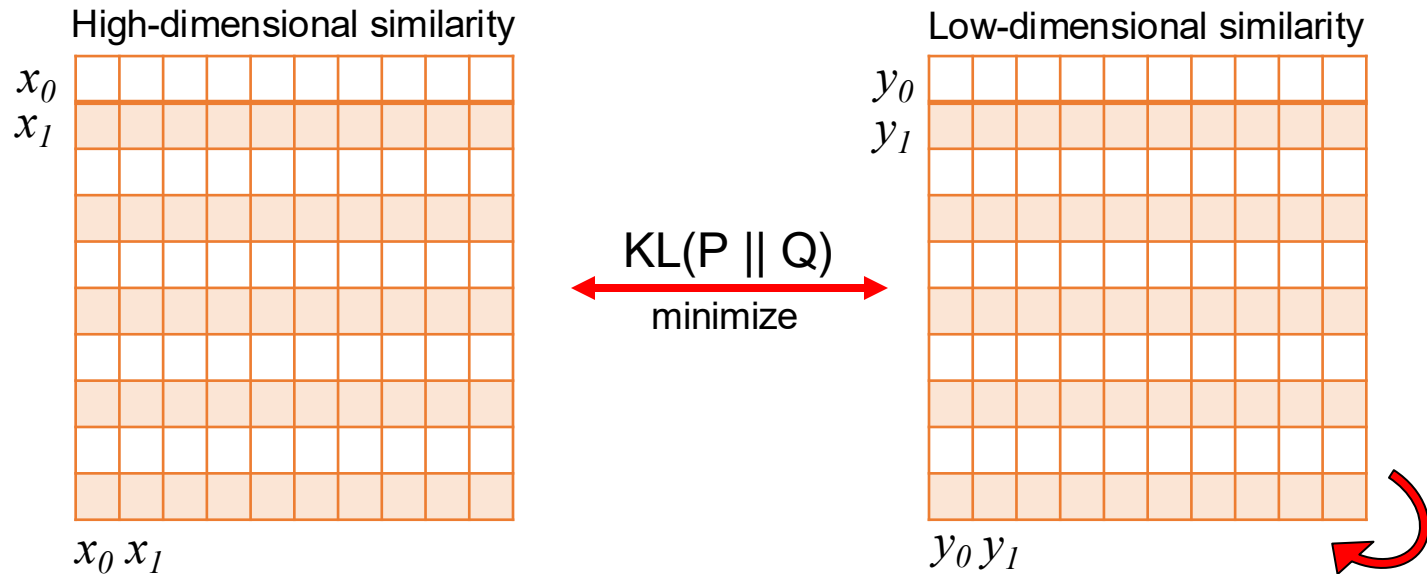
Comparing Two Probability Distributions

- Minimizing the sum of the Kullback-Leibler divergences between the conditional probabilities
- Kullback–Leibler divergence is a type of statistical distance: how one distribution P is different from a second distribution Q

$$\text{KL}(P \parallel Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$



Comparing Two Probability Distributions

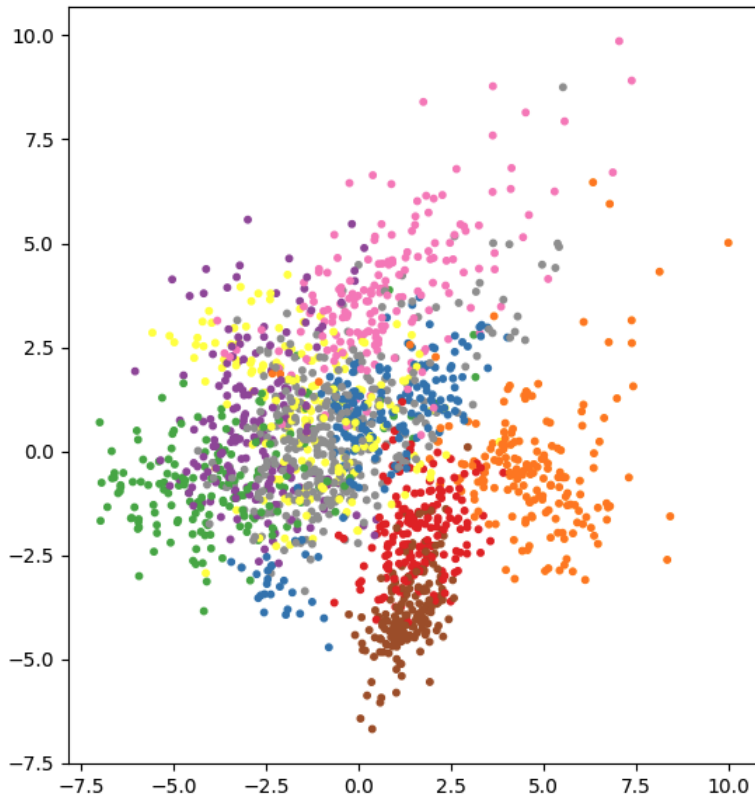


- P is fixed: similarities in high-dimensional space
- Q is flexible: similarities in low-dimensional space
 - depends on location of points in low-dimensional space
- Gradient descent: move points in direction that minimizes KL
- Local optimum

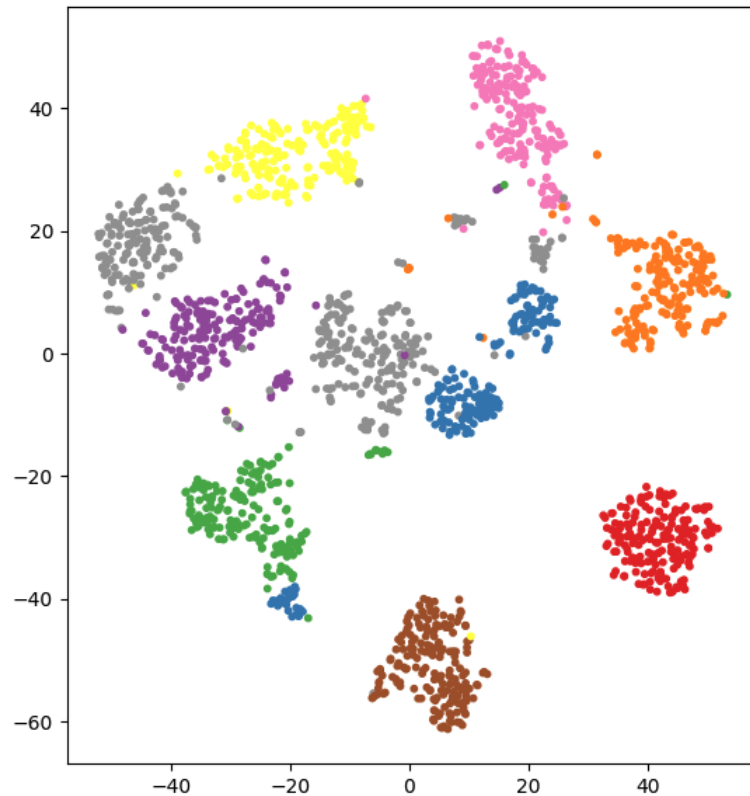
t-SNE on MNIST Dataset

- Hand-written digit recognition
- 28x28 pixels
- 784 dimensions

PCA



tSNE

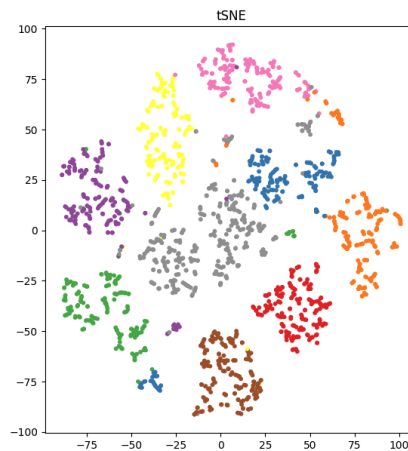


0 0 0 0 0 0 0 0 0
1 1 1 1 1 1 1 1 1
2 2 2 2 2 2 2 2 2
3 3 3 3 3 3 3 3 3
4 4 4 4 4 4 4 4 4
5 5 5 5 5 5 5 5 5
6 6 6
7 7 7
8 8 8
9 9 9

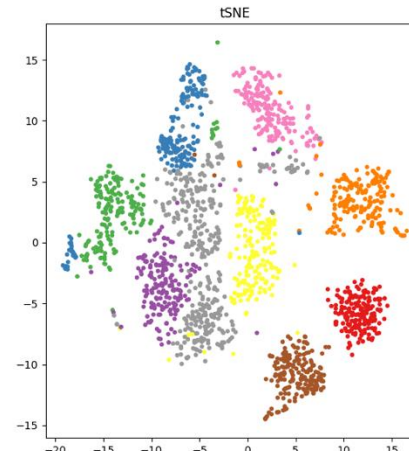
Perplexity

- Perplexity affects balance between local and global structure
- It defines the number of effective neighbours
- Small Perplexity (5-30):
 - fewer points as neighbours, focusing more on very local clusters
 - can miss the larger global structure
- Large Perplexity (e.g., 30-100):
 - more points are considered as neighbours, giving a more global view of the dataset
 - broader patterns, might smooth out finer details

perplexity=5.0



perplexity=200.0



Properties

- Non-linear mapping
- Close points end up close in low-dimensional space
- Proximity in low-dimensional space is no guarantee for original proximity
- Good separation between clusters
- No function is learned
 - 'only on the training set'
- Local optimum
 - re-run with different seed

Sklearn Implementation

sklearn.manifold.TSNE

<code>n_components, default=2</code>	Dimension of the embedded space
<code>perplexity, default=30.0</code>	Related to number of nearest neighbours
<code>max_iter, default=None</code>	Max number of iterations for optimisation
<code>init{'random', 'pca'}</code>	Initialisation of embedding
<code>random_state, default=None</code>	Determines the random number generator

e.g.

```
X, y = datasets.load_digits(return_X_y=True)
tsne = make_pipeline(StandardScaler(), TSNE(n_components=2, init='pca'))
tsne.fit(X, y)
X_tsne_2d = tsne.fit_transform(X)
plt.scatter(X_tsne_2d[:, 0], X_tsne_2d[:, 1])
```